2019 IEEE 27th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)

# HyperMapper: a Practical Design Space Exploration Framework

Luigi Nardi*, Artur Souza†, David Koeplinger‡ and Kunle Olukotun§
Stanford University, UFMG
*lnardi@stanford.edu, †arturluis@dcc.ufmg.br, ‡dkoeplin@stanford.edu, §kunle@stanford.edu

Design problems are ubiquitous in scientific and industrial achievements. Scientists design experiments to gain insights into physical and social phenomena, and engineers design machines to execute tasks more efficiently. These design problems are fraught with choices which are often complex and high-dimensional and which include interactions that make them difficult for individuals to reason about. In software/hardware co-design, for example, companies develop libraries with tens or hundreds of free choices and parameters that interact in complex ways. In fact, the level of complexity is often so high that it becomes impossible to find domain experts capable of tuning these libraries [1].

Typically, a human developer that wishes to tune a computer system will try some of the options and get an insight of the response surface of the software. They will start to fit a model in their head of how the software responds to the different choices. However, fitting a complex multi-objective function without the use of an automated system is a daunting task. When the response surface is complex, e.g. non-linear, non-convex, discontinuous, or multi-modal, a human designer will hardly be able to model this complex process, ultimately missing the opportunity of delivering high performance products.

Mathematically, in the mono-objective formulation, we consider the problem of finding a global minimizer of an unknown (black-box) objective function $f$:

$$x^* = \arg\min_{x \in \mathbb{X}} f(x) \quad (1)$$

where $\mathbb{X}$ is some input decision space of interest (also called design space). The problem addressed in this paper is the optimization of a deterministic function $f : \mathbb{X} \to \mathbb{R}$ over a domain of interest that includes lower and upper bound constraints on the problem variables.

When optimizing a smooth function, it is well known that useful information is contained in the function gradient/derivatives which can be leveraged, for instance, by first order methods. The derivatives are often computed by hand-coding, by automatic differentiation, or by finite differences. However, there are situations where such first-order information is not available or even not well defined. Typically, this is the case for computer systems workloads that include many discrete variables, *i.e.,* either categorical (*e.g.,* boolean) or ordinal (*e.g.,* choice of cache sizes), over which derivatives cannot even be defined. Hence, we assume in our applications of interest that the derivative of $f$ is neither symbolically nor numerically available. This problem is referred to in the literature as DFO [1], also known as black-box optimization [2] and, in the computer systems community, as design space exploration (DSE) [4].

In addition to objective function evaluations, many optimization programs have similarly expensive evaluations of constraint functions. The set of points where such constraints are satisfied is referred to as the *feasibility* set. For example, in computer micro-architecture, fine-tuning the particular specifications of a CPU (*e.g.,* L1-Cache size, branch predictor range, and cycle time) need to be carefully balanced to optimize CPU speed, while keeping the power usage strictly within a pre-specified budget. A similar example is in creating hardware designs for field-programmable gate arrays (FPGAs). Any generated FPGA design must keep the number of units strictly below this resource budget to be implementable on the FPGA. In these examples, feasibility of an experiment cannot be checked prior to termination of the experiment; this is often referred to as *unknown feasibility* in the literature [3]. Also note that the smaller the feasible region, the harder it is to check if an experiment is feasible (and even more costly to check optimality [3]).

Real-world engineering problems commonly have multiple objectives that have to be tuned simultaneously. The trade-off Pareto front resulting form the tuning is used for decision making and it is a tool for selecting the best trade-off for any specific user scenario. Specifically, multi-objective optimization is a crucial matter in computer systems design space exploration because real-world applications often rely on a trade-off between several objectives such as throughput, latency, memory usage, area, etc. A pictorial representation of a multi-objective problem is shown in Figure 1. On the left, a three-dimensional design space is composed by one ordinal ($p_1$), one real ($p_2$), and one categorical ($p_3$) variable. The red dots represent samples from this search space. The multi-objective function $f$ maps this input space to the output space on the right, also called the optimization space. The optimization space is composed by two optimization objectives ($o_1$ and $o_2$). The blue dots correspond to the red dots in the left via the application of $f$. The arrows Min and Max represent the fact that we can minimize or maximize the objectives as a function of the application. Optimization will drive the search of optima points towards either the Min or Max of the right plot.

While the growing demand for sophisticated DFO methods has triggered the development of a wide range of approaches

2375-0227/19/$31.00 ©2019 IEEE
DOI 10.1109/MASCOTS.2019.00053

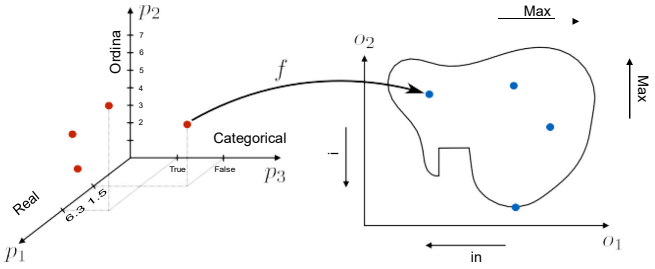


Fig. 1: Example of a multi-objective space. The multi-objective function $f$ maps each point in the 3-dimensional design space on the left to the optimization space on the right.

| Name | Multi | RIOC var. | Constr. | Prior |
|---|---|---|---|---|
| GpyOpt | | | | |
| OpenTuner | | ✓ | | |
| SURF | | ✓ | | |
| SMAC | | ✓ | | |
| Spearmint | | | ✓ | |
| Hyperopt | | ✓ | | ✓ |
| Hyperband | | ✓ | | |
| GPflowOpt | ✓ | | ✓ | |
| cBO | | | ✓ | |
| BOHB | | ✓ | | |
| HyperMapper 1.0 | ✓ | ✓ | | |
| **HyperMapper 2.0** | ✓ | ✓ | ✓ | ✓ |

TABLE I: Derivative-free optimization software taxonomy. *Multi* notes if the software is multi-objective or not. *RIOC var.* says if the software supports all Real, Int, Ordinal and Categorical variables. *Constr.* refers to inequality constraints that define a feasible region that are used in the optimization process. *Prior* represents the ability of the software to inject prior knowledge in the search.

and frameworks, none to date are featured enough to fully address the complexities of design space exploration and optimization in the computer systems domain. To address this problem, we introduce a new methodology and a framework dubbed HyperMapper 2.0. HyperMapper 2.0 is designed for the computer systems community and can handle design spaces consisting of multiple objectives and categorical/ordinal variables. Emphasis is on exploiting user prior knowledge via modeling of the design space parameters distributions. Given the years of hand-tuning experience in optimizing hardware, designers bear a high level of confidence. HyperMapper 2.0 gives means to inject knowledge in the search algorithm. This is achieved by introducing for the first time the use of a Beta distribution for modeling the user belief, i.e., prior knowledge, on how each parameter of the design space influences a response surface. In addition, bearing in mind the feasibility constraints that are common in computer systems workloads we introduce for the first time a model that considers unknown constraints, which is, constraints that are only known after evaluating a system configuration. To aid comparison, we provide a list of existing tools and the corresponding taxonomy in Table I. Our framework uses a model-based algorithm, i.e., construct and utilize a surrogate model of $f$ to guide the search process. A key advantage of having a model, and more specifically a white-box model, is that the final surrogate of $f$ can be analyzed by the user to understand the space and learn fundamental properties of the application of interest.

As shown in Table I, HyperMapper 2.0 is the only framework to provide all the features needed for a practical design space exploration software in computer systems applications.

## I. Demo

We introduce a new methodology and corresponding software framework, HyperMapper 2.0, which handles multi-objective optimization, unknown feasibility constraints, and categorical/ordinal variables. This new methodology also supports injection of the user prior knowledge in the search when available. All of these features are common requirements in computer systems but rarely exposed in existing design space exploration systems. The proposed methodology follows a white-box model which is simple to understand and interpret (unlike, for example, neural networks) and can be used by the user to better understand the results of the automatic search.

The paper [5] applies and evaluates the new methodology to the automatic static tuning of hardware accelerators within the recently introduced Spatial programming language, with minimization of design run-time and compute logic under the constraint of the design fitting in a target field-programmable gate array chip. Our results show that HyperMapper 2.0 provides better Pareto fronts compared to state-of-the-art baselines, with better or competitive hypervolume indicator and with 8x improvement in sampling budget for most of the benchmarks explored.

In this demo we will introduce the HyperMapper 2.0 framework and give comprehensive examples of the features that allow computer system engineers to easily plug-and-play this tool to automatically tune their toolchain. The demo will focus on:

- A methodology for multi-objective optimization that deals with categorical and ordinal variables, unknown constraints, and exploitation of the user prior knowledge.
- An integration and experimental results of our methodology in a full, production-level compiler toolchain for hardware accelerator design.
- A framework dubbed HyperMapper 2.0 implementing the newly introduced methodology, designed to be simple, user-friendly and application independent.

## References


[1] Andrew R Conn, Katya Scheinberg, and Luis N Vicente. *Introduction to derivative-free optimization*, volume 8. Siam, 2009.

[2] Paul Feliot, Julien Bect, and Emmanuel Vazquez. A bayesian approach to constrained single-and multi-objective optimization. *Journal of Global Optimization*, 67(1-2):97–133, 2017.

[3] Michael A Gelbart, Jasper Snoek, and Ryan P Adams. Bayesian optimization with unknown constraints. *arXiv preprint arXiv:1403.5607*, 2014.

[4] Engin İpek, Sally A McKee, Rich Caruana, Bronis R de Supinski, and Martin Schulz. *Efficiently exploring architectural design spaces via predictive modeling*, volume 41. ACM, 2006.

[5] L. Nardi, D. Koeplinger, and K. Olukotun. Practical design space exploration. *arXiv preprint arXiv:1810.05236*, 2018.